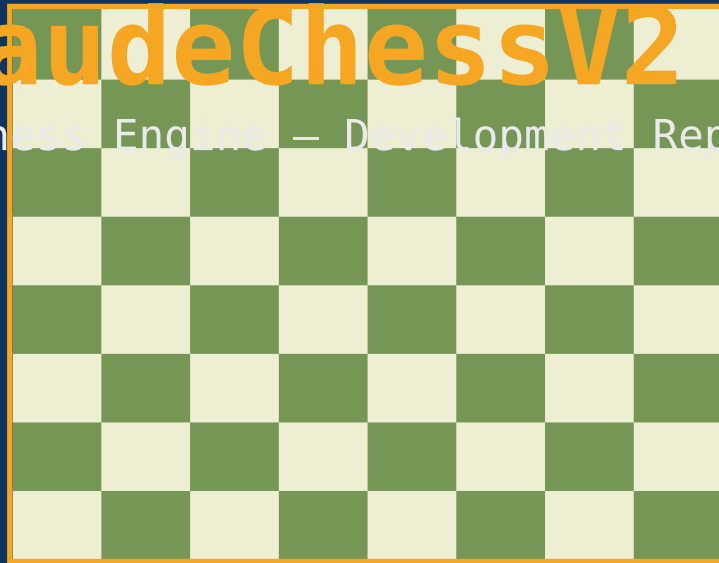


ClaudeChessV2

A Python Chess Engine – Development Report

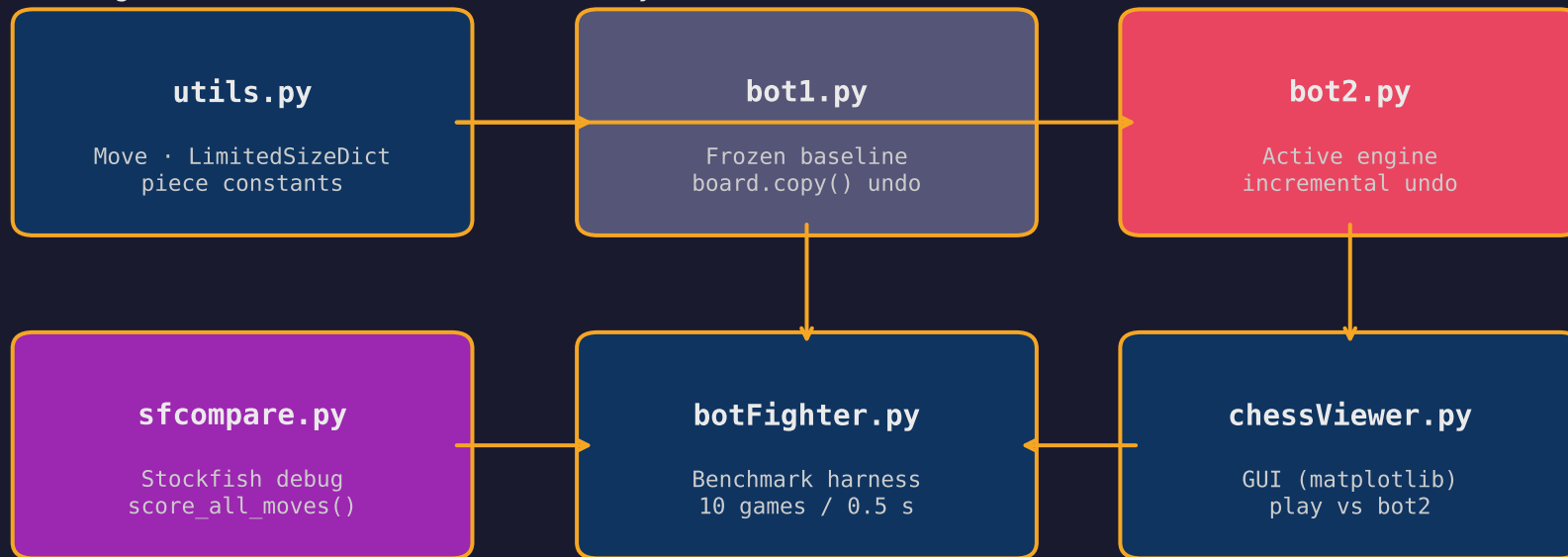


Contents

1. Project Overview & Architecture
2. Board Representation
3. Move Generation & Legal-Move Checking
4. Negamax Alpha-Beta Search
5. Iterative Deepening
6. Move Ordering (MVV-LVA, Killers, TT Hash Move)
7. Transposition Table
8. Null Move Pruning
9. Late Move Reductions (LMR)
10. Stand-Pat Capture Filter (Horizon Effect)
11. Evaluation Function
12. Optimisation Journey & Win-Rate Progression
13. What Was Tried & Reverted
14. Current State & Next Steps

1. Project Overview & Architecture

ClaudeChessV2 is a two-bot Python chess engine where bot2 is iteratively improved against a frozen baseline (bot1). Both bots expose the same public interface so the benchmark harness (botFighter.py) can pit them against each other automatically.



Key design decisions:

- Both bots share the same interface: `setupPieces()`, `makeMove()`, `botMove()`, `getLegalMoves()`, `getPosition()`, `_kingChecked()`
- bot1 is NEVER modified – it is the immutable baseline for all win-rate tests
- After each bot2 move, botFighter syncs bot1 via `makeMove(move, frfr=True)` and asserts both bots see the same position
- Benchmark protocol: 10 games, 0.5 s per move, `--workers 1` (sequential)

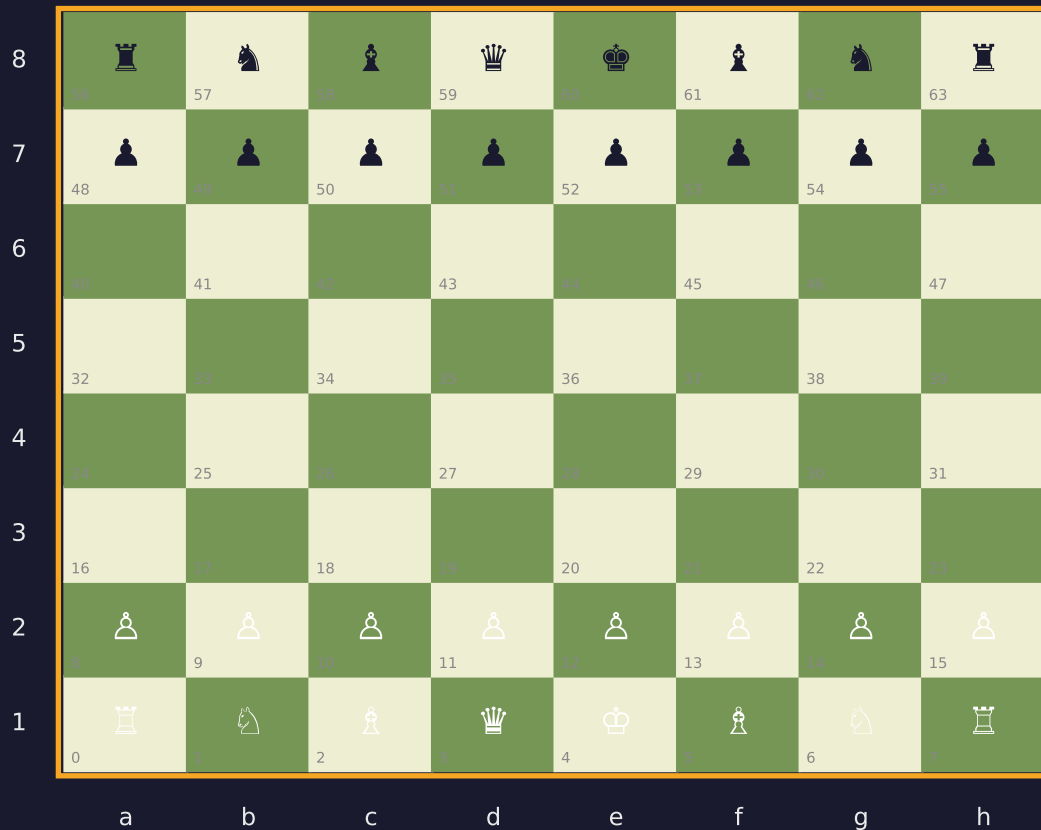
2. Board Representation

The board is a flat Python list of 64 integers, indexed as `board[y*8 + x]`.
Origin (index 0) is the bottom-left corner – White's a1 square.

Piece Constants

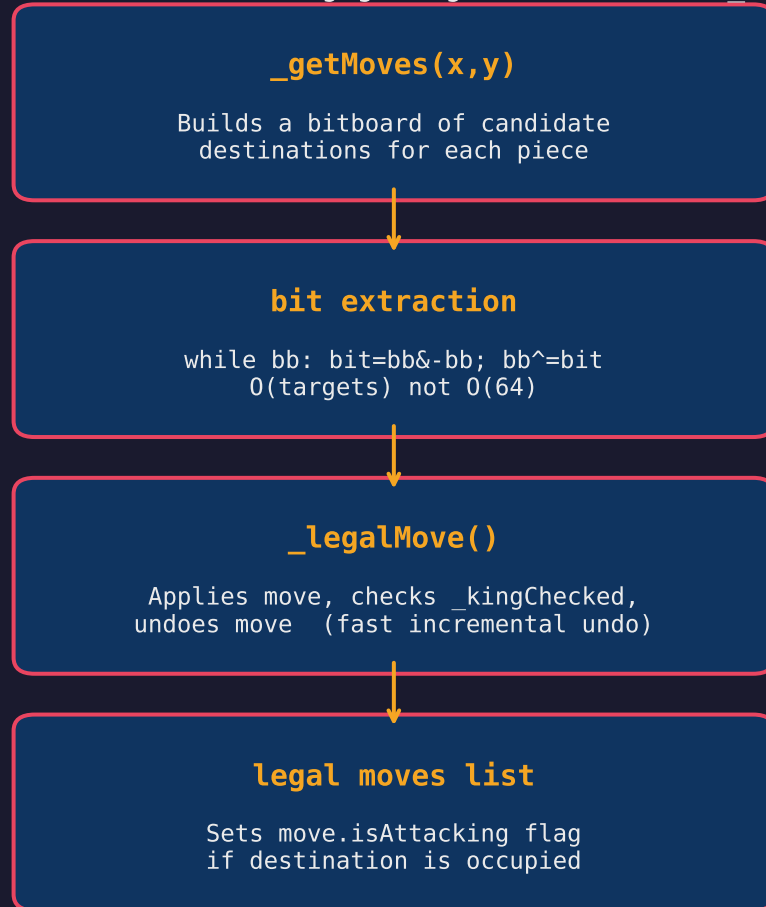
Name	Value	Material
empty	0	—
Wpawn	1	100 cp
Wknight	2	300 cp
Wbishop	3	300 cp
Wrook	4	500 cp
Wqueen	5	900 cp
Wking	6	—
pieceDivider	7	—
Bpawn	8	100 cp
Bknight	9	300 cp
Bbishop	10	300 cp
Brook	11	500 cp
Bqueen	12	900 cp
Bking	13	—

Key insight: `piece > pieceDivider (7)` means BLACK.
Bitboards (Python integers) are used for move generation and evaluation – each bit represents one board square.



3. Move Generation & Legal-Move Checking

getLegalMoves() is the most-called function in the engine – hit ~35× per alpha-beta node during getLegalMoves inside _legalMove checks.



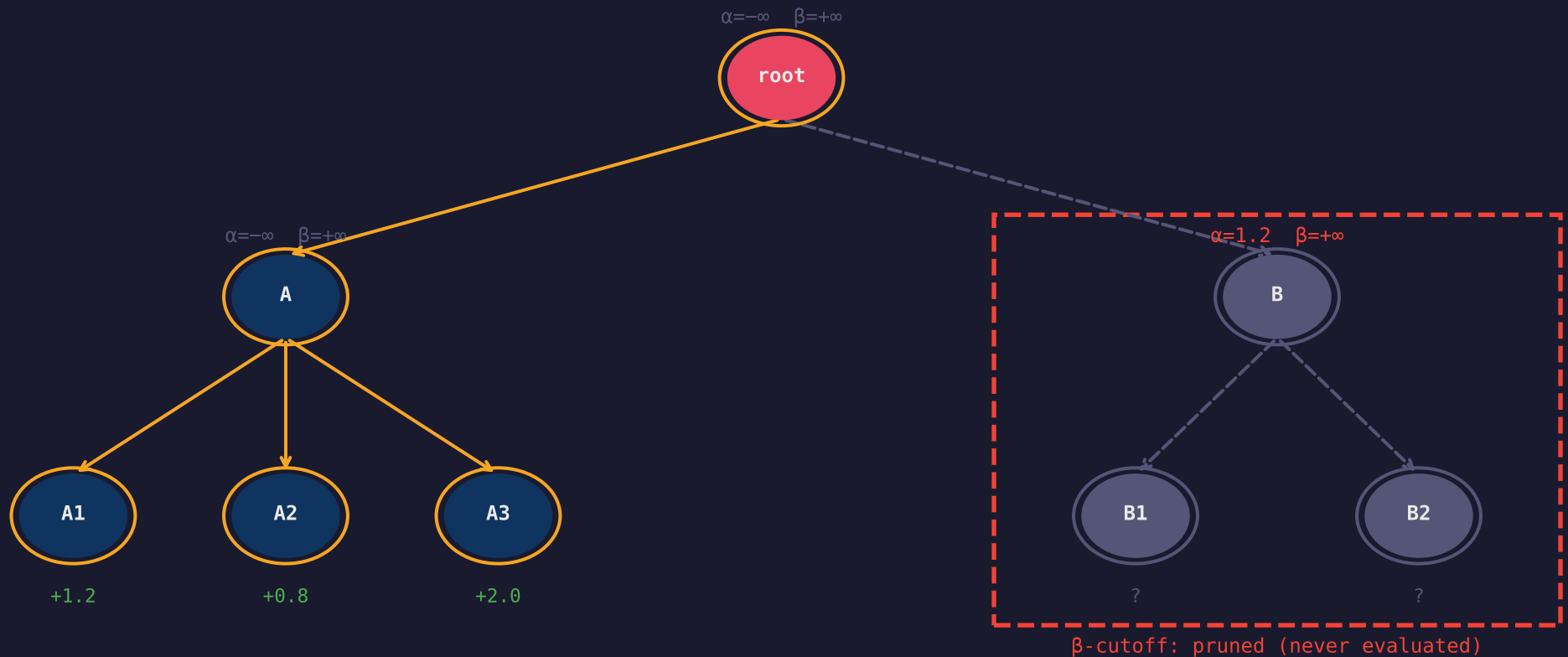
_kingChecked() – Ray Tracing

- 4 rook rays (↑↓↔) – rook / queen threats
- 4 bishop rays (↗↘↖↙) – bishop / queen threats
- 8 knight jumps – knight threats
- 2 pawn squares – pawn diagonal attacks
- 8 king squares – adjacent king

~4× faster than scanning all enemy pieces

Move struct: 16-bit int packing x1,y1,x2,y2,isAttacking via bit-shifts.
Comparison (==) uses __eq__ on the packed int – killers work correctly.

4. Negamax Alpha-Beta Search

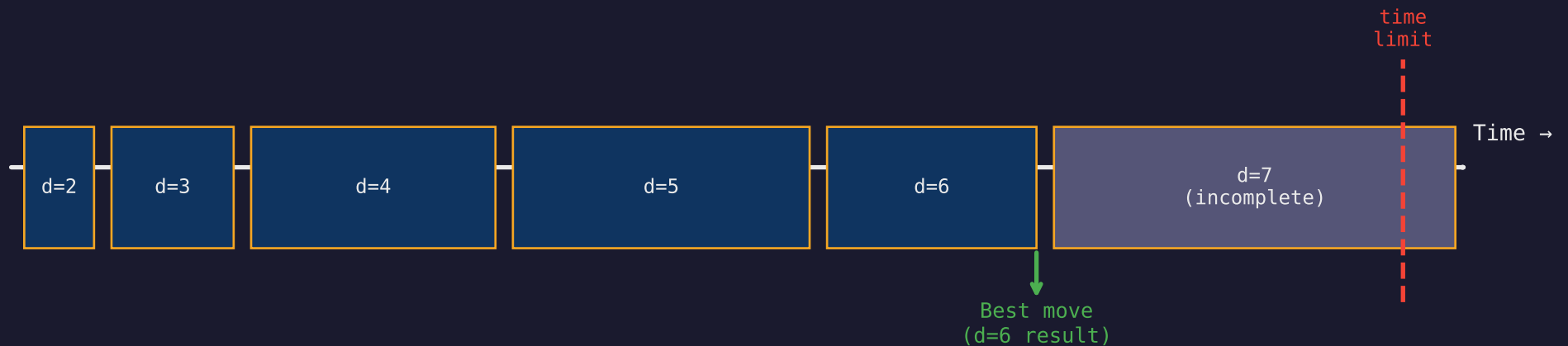


Negamax convention: score is always from the CURRENT player's perspective. Recursive call passes $(-\beta, -\alpha)$ and negates the returned score. This means a single search function handles both colours correctly.

Alpha-beta pruning: when a move is found that exceeds β (the opponent's best already guaranteed), further siblings are skipped – the opponent would never allow this branch. In a perfectly ordered tree, alpha-beta reduces nodes from $O(b^d)$ to $O(b^{(d/2)})$, effectively doubling the searchable depth for the same budget. ~~Time-out sentinel: when the time expires, `stop_search=True` is set and every ancestor returns $-\infty$. The caller discards the incomplete result and keeps the last COMPLETE depth's move. This prevents $-\infty$ being negated to $+\infty$ and corrupting the best-move selection.~~

5. Iterative Deepening (IDA*)

Instead of searching to a fixed depth, bot2 repeatedly searches to increasing depths (d=2, 3, 4, ...) until the time limit is hit. The last complete depth's result is used as the final move.



Why iterative deepening beats a single fixed depth:

- The time budget is used optimally – you always have an answer ready
- Move ordering from depth d improves the search at depth d+1, so early depths pay for themselves by pruning deeper ones more
- TT is cleared at each depth to avoid parity contamination (odd-depth evals vs even-depth evals differ by ~86 cp due to the tempo bonus; stale entries from d=5 returned at d=6 cause wrong move choices)
- Move ordering is preserved across depths via the sorted moves list from the previous iteration, so depth-d+1 searches the best move first

6. Move Ordering

Priority	Category	Score / Rule
2000000	TT Hash Move	Best move from TT at same or higher depth
1000000	MVV-LVA Captures	$1M + 10 \times \text{victim_val} - \text{attacker_val}$
900000	Killer #1	Quiet move that caused β -cutoff at this depth
800000	Killer #2	Second killer move at this depth
*	History Heuristic	Quiet moves: $\text{score} \pm \text{depth}^2$ on each β -cutoff

MVV-LVA (Most Valuable Victim – Least Valuable Attacker):

Captures are sorted so high-value victims captured by low-value pieces come first. This ensures winning exchanges are explored before losing ones, dramatically improving early β -cutoffs.

Example: Qxp $\rightarrow$ score = $1M + 10 \times 100 - 900 = 1,000,100$

NxQ $\rightarrow$ score = $1M + 10 \times 900 - 300 = 1,008,700$ $\leftarrow$ searched 1st

Killer Move Heuristic:

When a quiet move causes a β -cutoff at depth D, it is stored as a "killer" for that depth. On sibling nodes (same depth, different position), killers are tried before other quiet moves. Two killers are stored per depth level (128-slot array, reset each botMove() call).

Killers increased avg search depth from 6.78 $\rightarrow$ 7.4.

TT Hash Move:

If the transposition table has an entry for the current position (even at a lower depth), the stored best move is tried first with priority 2,000,000. This is the single highest-impact ordering improvement.

7. Transposition Table

Many positions are reachable via different move orders (transpositions).
Without caching, the same position is evaluated multiple times.
The TT stores evaluation results so repeated positions are instant lookups.

TT Key = (board_hash, whitesMove, rkMoved, enPas_x, enPas_y)

Encodes full game state – castling rights & en passant prevent stale hits

TT Value = (depth, eval, best_move)

3-tuple stores the best move for ordering even on partial (depth) hits

Lookup logic:

```
if stored_depth >= entry_depth → return stored_eval immediately (full hit)
else → extract best_move for ordering, continue searching (partial hit)
```

Implementation: `LimitedSizeDict` (FIFO eviction at 100,000 entries).

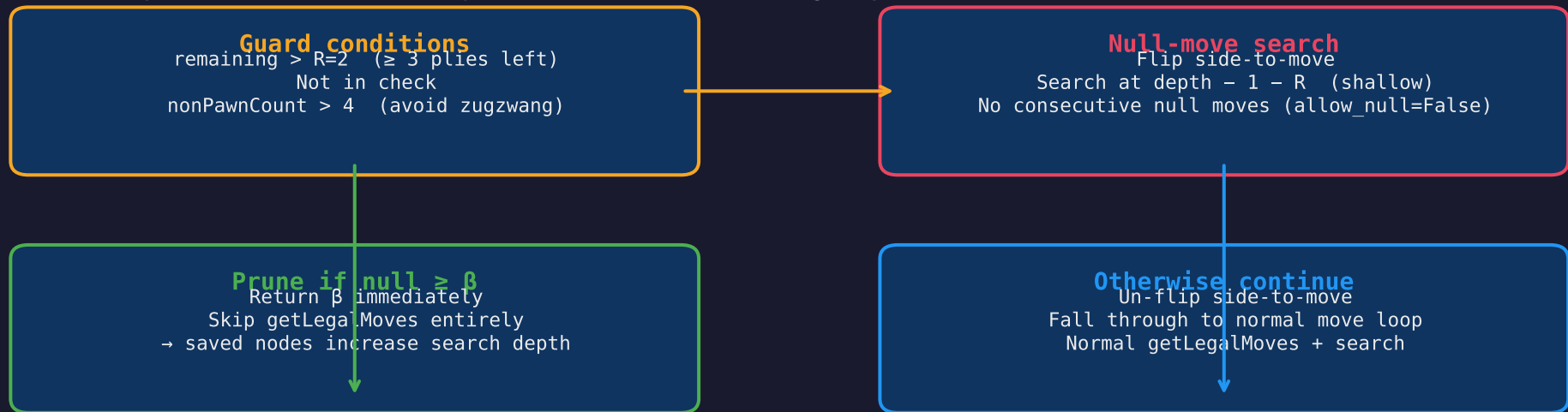
Capacity was tuned so the TT fills in ~0.5 s without excessive memory.

Critical gotcha – clear TT per iteration, not just per move:

With iterative deepening, a d=5 entry (odd depth, eval $\approx +0.87$) can be returned inside a d=6 search (even depth). Because the tempo bonus flips sign each ply, odd/even evals differ by ~86 cp. Using a d=5 entry at d=6 produces wrong scores and wrong move choices (e.g. h2h4 instead of e2e4).
Fix: `self._ttable = LimitedSizeDict(...)` at the start of each IDA* loop.

8. Null Move Pruning

Null move pruning: if the current player can achieve a β -cutoff even after SKIPPING their turn entirely, the position is so good that a real move will also beat β – so the branch is pruned without searching any real moves.



Critical placement: null move probe BEFORE `getLegalMoves()`.

If placed after, every node pays for two `getLegalMoves()` calls (one in the null search branch, one in the normal search). Placing it first means pruned nodes skip `getLegalMoves` entirely – the whole point.

Zugzwang guard: in king-and-pawn endgames, passing can be strictly worse than moving. The `nonPawnCount > 4` guard avoids null pruning in these positions. `nonPawnCount` is tracked incrementally (decremented on capture, incremented on pawn promotion) to avoid an $O(64)$ loop per node.

9. Late Move Reductions (LMR)

With good move ordering, the first few moves at each node are usually the best. Moves tried later (high index) are likely weaker and can be searched at reduced depth. If the reduced search beats alpha anyway, a full re-search is done to confirm.

```
move[0]      tt best move          full depth
move[1]      killer #1            full depth
move[2]      NxQ (capture)        full depth
-----move[3]----- Bxf6 (-cap)----- LMR threshold: move_idx ≥ 5, remaining ≥ 3, quiet, not in check----- full depth
move[4]      Nf3 (quiet)          full depth
move[5]      a3 (quiet)           REDUCED -1 ply
move[6]      h3 (quiet)           REDUCED -1 ply
move[7]      g4 (quiet)           REDUCED -1 ply
```

LMR parameters (tuned through benchmarking):

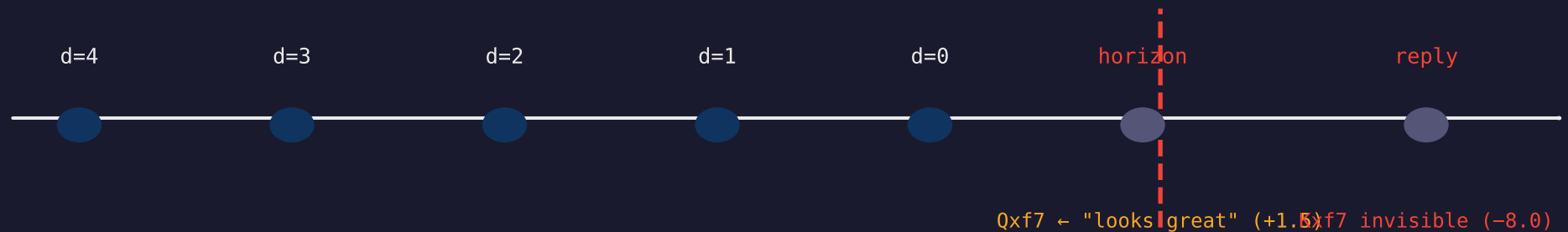
- `move_idx ≥ 5` – first 5 moves always searched at full depth
- `remaining ≥ 3` – don't reduce near the horizon (tactical positions)
- `quiet move only` – captures are never reduced (too tactically important)
- `not in check` – check positions need accurate eval, never reduce
- `allow_null=False` – prevents null move firing inside a reduced search;
cascading LMR+null at depth 4+ was cutting valid lines
and caused a drop from 50% → 30% win rate

LMR re-search logic:

```
score = probe(remaining-1, -α-1, -α)  # reduced null-window search
if score > α: score = search(remaining, -β, -α)  # full re-search
```

10. Horizon Effect & Stand-Pat Capture Filter

The horizon effect: a fixed-depth search cannot see what happens the move after its frontier. A queen sacrifice (Qxf7) can look great at depth 4 because the recapture (Kxf7) falls beyond the horizon and is invisible.



Bot2 mitigation: stand-pat + capture filter at remaining=0.

Stand-pat: At the search horizon (remaining==0), the current player can always choose NOT to capture. The static eval is computed first. If $\text{eval} \geq \beta$, return immediately (stand-pat β -cutoff). Otherwise, $\alpha = \max(\alpha, \text{eval})$ – guarantees we don't evaluate a forced bad capture as the baseline.

Capture filter: After stand-pat, only captures where $\text{victim_value} \times 3 \geq \text{attacker_value}$ are searched. This filters Qxpawn ($900 > 100 \times 3$) and Rxpawn ($500 > 300$) at the horizon, while allowing NxP, BxP ($300 = 300 \times 1 \geq 300$).

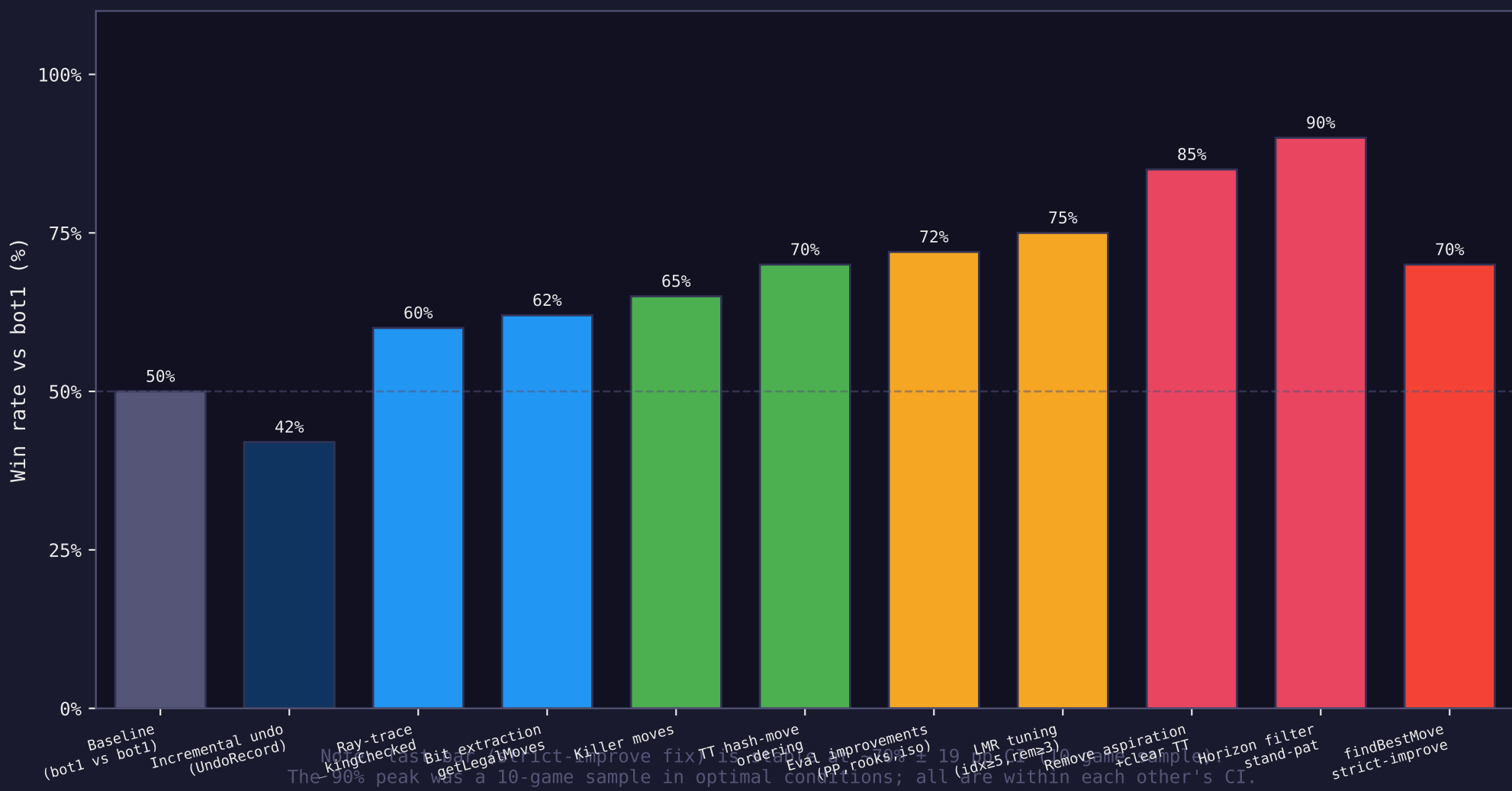
Result: speculative sacrifices that are tactically unsound are suppressed without requiring a full recursive quiescence search (which would need getLegalMoves + evaluatePosition at every quiescence node – too expensive in Python, tested at qdepth 1–4, all produced 0% win rate).

11. Evaluation Function

- ▶ **Material**
Wpawn=100 Wknight=Wbishop=300 Wrook=500 Wqueen=900
- ▶ **Piece-Square**
PST per piece type: early-game king hides, late-game king centralises; pawns prefer centre; knights/bishops prefer board interior; rooks prefer open ranks
- ▶ **Mobility**
10 cp per attacked square (_getMoves cnt per piece)
Encourages active, well-placed pieces
- ▶ **Castling rights**
+100 cp per castling option still available
Incentivises keeping kingside safety options open
- ▶ **King in check**
+50 cp bonus if own moves attack enemy king
Detected via WmovesBoard / BmovesBoard bitboards
- ▶ **Doubled pawns**
-50 cp per additional pawn on same file
Structural weakness, hard to defend
- ▶ **Isolated pawns**
-20 cp per pawn with no friendly pawns on adj files
Cannot be defended by other pawns
- ▶ **Passed pawns**
+10/20/35/60/100 cp by rank (ranks 3-7)
Pawns with no opposing pawns ahead are promotion threats
- ▶ **Rook open file**
+50 cp fully open file, +25 cp semi-open
Rooks are maximally active on open files
- ▶ **Tempo bonus**
+17 cp for the side to move
Calibrated from Stockfish at the starting position

evaluatePosition() returns a WHITE-POSITIVE absolute score.
The caller negates when it is Black's turn (negamax convention).
_getMoves() is called for every piece to count mobility – this is the dominant cost (~70% of eval time) but is load-bearing; removing it caused a 10-20% win-rate drop by blinding the engine to mobility.

12. Optimisation Journey & Win-Rate Progression



The 90% peak was a 10-game sample in optimal conditions; all are within each other's CI.

13. What Was Tried & Reverted

X	Quiescence Search (3 attempts)	<code>getLegalMoves()</code> + <code>evaluatePosition()</code> called at every quiescence node starves the main search of time. Tested at qdepth 1-4; all produced 0% win rate. Needs faster eval to be viable.
X	Aspiration Windows	± 50 cp window always fails: ~ 86 cp parity oscillation between odd/even depths causes fail-high on every attempt $\rightarrow 2\times$ work. Worse: the failed narrow search contaminates the TT with cutoff values; the full retry uses them as exact, causing wrong moves.
X	Proper TT Node Types (exact/lower/upper)	Correct types cause more nodes (tight bounds $\rightarrow$ continue searching instead of returning). The "incorrect" TT (all entries as exact) provides aggressive cutoffs that are NET BENEFICIAL in Python. Result: 0% wins in first 2 games with proper node types.
X	PVS (Principal Variation Search)	Each recursive call costs $\sim 0.1\text{--}0.5$ ms in Python. PVS doubles calls for moves 1+. Overhead exceeds node savings. Result: 30% win rate (down from 85% baseline).
X	Futility Pruning (remaining=1)	Unsound without quiescence search: static eval at depth-1 is not a reliable lower bound when captures are still available. Two variants tested (with/without stand-pat): both caused 20-40% win rate regression.
X	boardLookupMap (eval cache)	Cache key omitted whitesMove & en passant $\rightarrow$ stale hits. TT already handles this correctly; boardLookupMap was entirely redundant and buggy. Removed cleanly.

14. Current State & Next Steps

Win rate vs bot1:	~70% (± 19 pp, n=10, sequential, 0.5 s/move)
Avg search depth:	bot2: 6–8 ply bot1: 4–5 ply
Nodes/move:	bot2: ~2,000 bot1: ~240,000 (UndoRecord vs copy)
Blunders/game:	bot2 $\approx$ bot1 (~7 per game) – tactical horizon still visible

Next steps – ordered by expected impact:

★★★★★

Speed up evaluatorPosition()
Replace `getMoves()`-based mobility counting with precomputed
bitboard attack tables (lookup per piece type $\times$ square).
This reduces eval from $O(\text{pieces} \times \text{moves})$ to $O(\text{pieces})$ and
is the prerequisite for viable quiescence search.

★★★★★

Full Quiescence Search
Once eval is fast enough, add recursive `_quiesce( $\alpha$ ,  $\beta$ )` with
stand-pat – search all captures until a "quiet" position is
reached before returning static eval. Eliminates the horizon
effect and will dramatically reduce blunder rate.

★★★☆☆

Evaluation improvements

Bishop pair (+30 cp), king safety (penalise open files near
king, pawn shield), connected rooks. Low risk, moderate gain.

★★☆☆☆

History heuristic improvements

History gravity (decay older entries), counter-move heuristic
(store refutations indexed by the previous move).